

Model Comparison: UNetClassic vs UNetClassicConditioned

Experiment Setup

- **Training datasets:** turbulent_radiative_layer_2D + active_matter
- **Test dataset:** helmholtz_staircase
- **Resolution:** 128×384
- **Input timesteps (T_i):** 4
- **Output timesteps (T_o):** 1
- **Optimizer:** AdamW
- **LR Scheduler:** LinearWarmupCosineAnnealingLR
- **Evaluation:** Multi-step rollout validation

Two models were compared:

1. **UNetClassic** (baseline, no conditioning)
2. **UNetClassicConditioned** (PDE text embedding conditioning)

The conditioned model augments the input with a projected PDE embedding derived from dataset metadata (field names, resolution, boundary conditions).

Overall Rollout Performance

Model	Rollout Test Loss
UNetClassic (without emd)	0.13627
UNetClassic (with emd)	0.15746
AViT (without emd)	0.71870
AViT (with emd)	0.48534

Observation: UNetClassic achieves the best performance. AViT significantly underperforms— $3.5\times$ to $5.3\times$ worse rollout loss than the baseline.

Field-wise Metrics (Example: D_{xx})

Metric	UNetClassic (without emd)	UNetClassicConditioned (with emd)	AViT (without emd)	AViT (with emd)
L_∞	0.10191	0.48775	1.95246	1.33724
MSE	0.00277	0.02653	0.32536	0.14104
RMSE	0.05264	0.16286	0.55099	0.35132
NRMSE	166.46	515.02	1742.38	1110.96

Observation:

- UNetClassic achieves the best metrics across all fields.
- UNetClassicConditioned shows degradation ($\sim 5\times$ worse MSE).
- Both AViT variants perform poorly; L_∞ error is 13–19 $\times$ higher than UNetClassic.
- AViT(train.py) worse than AViT(multi_pde) suggests the conditioned variant behaves better in single-dataset mode.

Model Capacity

Model	Parameter Norm
UNetClassic (without emd)	2597.11
UNetClassicConditioned (with emd)	2923.88
AViT (without emd)	3854.95
AViT (with emd)	3869.67

AViT models have significantly higher parameter norms ($\sim 50\%$ more than UNet variants), yet achieve worse performance. This suggests capacity alone does not determine model suitability for this task.

Interpretation

1. Conditioning is Stable but Not Beneficial

- PDE text conditioning does not degrade training stability.
 - However, after 20 epochs of training, it significantly degrades rollout accuracy.
 - The conditioned model shows $\sim 3.8\times$ higher L_∞ error and $\sim 10\times$ higher MSE on the D_{xx} field.
 - The baseline model achieves superior performance across all metrics.
-

2. Why Conditioning Underperforms

- **Channel padding provides implicit dataset identity:** The multi-dataset architecture already encodes dataset information through padded channel structure; additional conditioning may be redundant or conflicting.
 - **Model overfitting to conditioning signal:** The conditioned model has 12% higher parameter norm, suggesting the embedding projection layers may overfit to the conditioning signal without improving generalization.
 - **Suboptimal conditioning design:** The current text-embedding-based conditioning may not align well with the information content needed by UNet.
 - **Opposite of intended effect:** Rather than improving OOD performance, conditioning appears to harm in-distribution accuracy, possibly by constraining model capacity in unfavorable ways.
-

3. Why AViT Severely Underperforms

- **Architectural mismatch:** AViT’s axial attention mechanism is optimized for very high-resolution spatial patterns (MPP-style), whereas `turbulent_radiative_layer + active_matter` at 128×384 may not benefit from this design.
- **Attention overhead without benefit:** The multi-head scaled-dot-product attention in AViT introduces significant computational overhead and hyperparameter sensitivity without corresponding accuracy gains.
- **Poor feature learning:** AViT’s hMLP embedding and embedding stems may compress temporal information less effectively than UNet’s hierarchical convolutions, especially for the 4-step input stacking.

- **Attention mechanisms struggle with PDE dynamics:** Vision transformers generally require more data and longer schedules to learn spatiotemporal correlation patterns. 20 epochs may be insufficient for attention heads to capture multi-scale PDE dynamics.
- **AViT(train.py) worse than AViT(multi_pde):** The single-dataset variant underperforms further, suggesting that handling multiple datasets in the multi_pde version—especially through naive channel concatenation—provides useful implicit regularization.

Summary Comparison

Performance Ranking (Best to Worst)

1. **UNetClassic (without embedding) — 0.13627** Best overall
2. UNetClassic (with embedding) — 0.15746 (↑ 15.5% worse)
3. AViT (with embedding) — 0.48534 (↑ 256% worse)
4. AViT (without embedding) — 0.71870 (↑ 427% worse)

Key Findings

Finding	Impact
Embedding hurts UNet	+15.5% test loss degradation
Embedding helps AViT	−32% test loss improvement (but still poor absolute performance)
UNet dominates AViT	5.3–3.5× better rollout loss
Capacity Performance	AViT has 48% more parameters but 3.5–5.3× higher loss
Multi-dataset training helps AViT	+48% relative improvement vs. single-dataset

Conclusions

1. **For this domain, UNetClassic without embedding is the clear winner:**
 - Lowest rollout loss (0.13627)
 - Best field-wise metrics (lowest L_∞ , MSE, RMSE, NRMSE)
 - Smallest parameter norm with highest accuracy (Pareto-efficient)
2. **Embedding conditioning is counterproductive for both architectures:**

- Degrades UNet: +15.5% rollout loss
- Required as a band-aid for AViT: −32% loss (still far from competitive)
- Suggests implicit multi-dataset handling via channel padding is sufficient

3. AViT is fundamentally unsuited for this task:

- 3.5–5.3× higher loss than UNetClassic
- Excels at very high-resolution vision tasks, not PDE rollout at 128×384
- Requires embedding to achieve any semblance of reasonable (but still poor) performance
- Even with conditioning, fails to approach UNet baseline performance

the-well-download --base-path "/Users/divyam/Course/Project Arbeit" --dataset active_matter

cd the_well/benchmark python3 train.py experiment=unet_classic server=local data=turbulent_radiative_layer_2D python3 train_multi_pde.py experiment=unet_classic_conditioned server=local data=turbulent_radiative_layer_2D

python3 train.py experiment=unet_classic server=local data=multi_dataset

epoch: 1 auto_resume: true/false